

Orange Digital Center

Formation Dev Data

DEV DATA P8

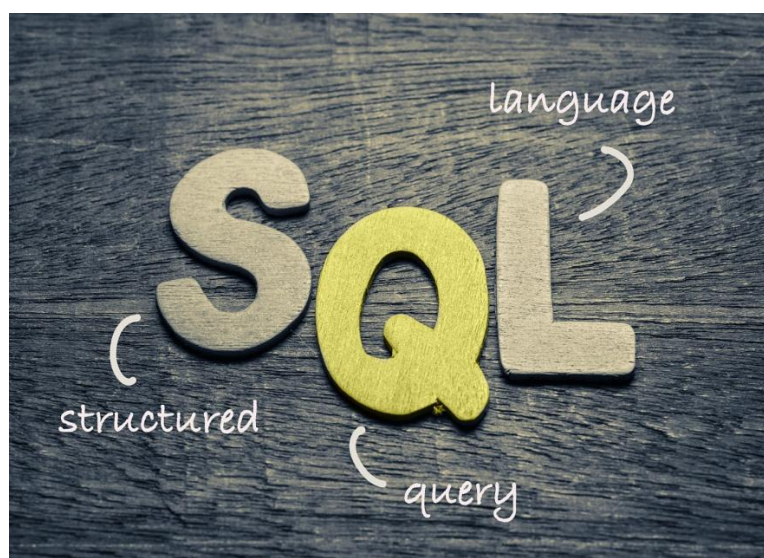
Bases de Données — Algèbre Relationnelle — SQL

Rapport de Projet Complet

Apprenante : Ndeye Penda SARR

Classe : DEV-DATA

Année : 2026 – 2027



1. Contexte et Présentation du Projet

Ce projet a pour objectif de concevoir et implémenter une base de données relationnelle pour une plateforme de formation. Il couvre l'ensemble du cycle de vie d'un projet data : de l'analyse du besoin métier jusqu'à l'écriture de requêtes SQL avancées en passant par la modélisation.

1.1 Énoncé du sujet

Le coach nous demande de gérer une plateforme de formation avec les entités suivantes :

- Des apprenants inscrits dans des classes
- Des modules d'enseignement organisés par catégorie
- Des inscriptions reliant les apprenants aux modules
- Des notes attribuées aux apprenants par module

1.2 Analyse approfondie du contexte métier

Avant d'écrire une seule ligne de code, nous avons analysé en profondeur le contexte pour choisir le bon modèle. Deux interprétations étaient possibles :

Interprétation A — Plateforme en ligne (type Coursera, Udey)

Les apprenants s'inscrivent librement aux modules de leur choix.

Pas de notion de classe ou de groupe.

Les évaluations et examens se font individuellement.

REJETÉE : le sujet mentionne explicitement des classes et des groupes.

Interprétation B — Établissement physique en cours de digitalisation (RETENUE)

Les apprenants appartiennent à une classe (DEV-DATA, DEV-WEB, DEV-IA).

Chaque classe a un programme prédéfini de modules.

Tous les apprenants d'une même classe avancent ensemble dans les modules.

Les groupes sont assignés par le formateur pour chaque module.

Les évaluations sont collectives : même type de note pour tous dans une classe.

RETENUE : cohérente avec les questions sur les classes, groupes et validations.

Cette analyse du contexte métier est fondamentale. Elle a guidé chaque décision de modélisation qui suit.

2. Modélisation — Partie 1

2.1 Réponses aux questions

Question 1 — Quelles sont les tables nécessaires ?

Nous avons identifié 6 tables. Voici la justification de chacune :

Table	Rôle	Justification
CLASSES	Entité principale	Une classe est une entité à part entière avec son propre programme. Normalisation : évite de répéter le nom de la classe comme VARCHAR partout.
MODULES	Entité principale	Un module est une entité indépendante avec ses propres caractéristiques (catégorie, durée).
CLASSE_MODULE	Association N-N	Une classe fait plusieurs modules, un module est fait par plusieurs classes. Table de liaison obligatoire. Contient l'état d'avancement collectif.
APPRENANTS	Entité principale	Chaque apprenant appartient à une classe.

INSCRIPTIONS	Association N-N	Matérialise l'acte individuel d'inscription. Distincte de CLASSE_MODULE car elle est individuelle.
NOTES	Entité dépendante	Un apprenant peut avoir plusieurs notes dans un même module (Devoir, Projet, Examen). Intentionnellement indépendante d'INSCRIPTIONS pour permettre la détection d'anomalies.

Question 2 — Quels sont les champs de chaque table ?

CLASSES (id, nom)

Champ	Type	Obligatoire	Justification
id	INT AUTO_INCREMENT	Oui	Clé primaire technique
nom	VARCHAR(50) UNIQUE	Oui	Nom de la classe, unique dans le système

MODULES (id, nom, categorie, duree_heures)

Champ	Type	Obligatoire	Justification
id	INT AUTO_INCREMENT	Oui	Clé primaire
nom	VARCHAR(100)	Oui	Nom du module
categorie	VARCHAR(100)	Oui	Permet de regrouper les modules par domaine
duree_heures	DECIMAL(5,1)	Oui	Durée totale du module en heures

CLASSE_MODULE (classe_id, module_id, etat)

Champ	Type	Obligatoire	Justification
classe_id	INT FK	Oui	Référence CLASSES — partie de la PK composite
module_id	INT FK	Oui	Référence MODULES — partie de la PK composite
etat	ENUM	Oui	Non commencé / En cours / Terminé — état collectif de la classe sur ce module

APPRENANTS (id, nom, prenom, email, sexe, adresse, classe_id, actif)

Champ	Type	Obligatoire	Justification
id	INT AUTO_INCREMENT	Oui	Clé primaire
nom / prenom	VARCHAR(100)	Oui	Identité civile
email	VARCHAR(150) UNIQUE	Oui	Identifiant unique de connexion
sexe	ENUM(M,F)	Oui	Genre
adresse	VARCHAR(255)	Non	Adresse physique
classe_id	INT FK	Oui	Lien vers la classe — un seul par apprenant
actif	BOOLEAN	Oui	Soft delete : FALSE si apprenant a quitté l'école (traçabilité)

INSCRIPTIONS (id, apprenant_id, module_id, date_inscription, groupe)

Champ	Type	Obligatoire	Justification
id	INT AUTO_INCREMENT	Oui	Clé primaire technique
apprenant_id	INT FK	Oui	Référence APPRENANTS
module_id	INT FK	Oui	Référence MODULES
date_inscription	DATE	Oui	Date administrative d'inscription individuelle
groupe	ENUM('G1','G2','G3','G4')	Oui	Groupe assigné par le formateur pour ce module spécifique

NOTES (id, valeur, type_note, apprenant_id, module_id)

Champ	Type	Obligatoire	Justification
id	INT AUTO_INCREMENT	Oui	Clé primaire
valeur	DECIMAL(5,2) CHECK (valeur BETWEEN 0 AND 20)	Oui	Note entre 0 et 20 — contrainte CHECK
type_note	ENUM	Oui	Devoir / Projet / Examen
apprenant_id	INT FK	Oui	Référence APPRENANTS
module_id	INT FK	Oui	Référence MODULES — pas d'inscription FK volontairement
date_evaluation	DATE NOT NULL	Oui	Date de l'évaluation permet les analyses temporelles

Question 3 — Quelles sont les clés primaires ?

Table	Clé primaire	Type
CLASSES	Id	Entier auto-incrémenté
MODULES	Id	Entier auto-incrémenté
CLASSE_MODULE	(classe_id, module_id)	Clé primaire composite — garantit unicité du couple
APPRENANTS	Id	Entier auto-incrémenté
INSCRIPTIONS	id + UNIQUE (apprenant_id, module_id)	Un apprenant ne peut s'inscrire qu'une fois par module
NOTES	Id	Entier auto-incrémenté — plusieurs notes possibles par apprenant/module

Question 4 — Quelles sont les relations entre les tables ?

Relation	Cardinalité	Explication
CLASSES → APPRENANTS	1,N	Une classe contient plusieurs apprenants. Un apprenant appartient à une et une seule classe.
CLASSES → MODULES	N,N	Une classe fait plusieurs modules. Un module est fait par plusieurs classes. Table CLASSE_MODULE.
APPRENANTS → MODULES	N,N	Un apprenant s'inscrit à plusieurs modules. Un module a plusieurs apprenants. Table INSCRIPTIONS.
APPRENANTS → NOTES	1,N	Un apprenant peut avoir plusieurs notes.
MODULES → NOTES	1,N	Un module peut avoir plusieurs notes associées.

Question 5 — Quelles sont les clés étrangères ?

Table	Clé étrangère	Référence	ON DELETE	ON UPDATE
CLASSE_MODULE	classe_id	CLASSES(id)	RESTRICT	CASCADE
CLASSE_MODULE	module_id	MODULES(id)	RESTRICT	CASCADE
APPRENANTS	classe_id	CLASSES(id)	RESTRICT	CASCADE
INSCRIPTIONS	apprenant_id	APPRENANTS(id)	RESTRICT	CASCADE
INSCRIPTIONS	module_id	MODULES(id)	RESTRICT	CASCADE
NOTES	apprenant_id	APPRENANTS(id)	RESTRICT	CASCADE
NOTES	module_id	MODULES(id)	RESTRICT	CASCADE

Pourquoi ON DELETE RESTRICT et non CASCADE ?

CASCADE supprime automatiquement les données liées quand on supprime un parent. RESTRICT bloque la suppression si des données liées existent, ce qui force une gestion manuelle.

Dans un établissement scolaire, on ne supprime jamais un apprenant qui part.

On utilise le champ 'actif = FALSE' (soft delete) pour désactiver sans perdre l'historique. Cela garantit la traçabilité des données : bulletins, attestations, statistiques de réussite.

2.3 Justification des choix de modélisation

Pourquoi une table CLASSES séparée ?

Sans table CLASSES, le nom de la classe serait un VARCHAR répété dans APPRENANTS et CLASSE_MODULE.

Cela créerait des risques de fautes de frappe et d'incohérences.

Avec une table CLASSES, la classe_id est un entier vérifié par FK — impossible d'avoir 'DEV-DATA' et 'DEV_DATA'.

Pourquoi etat dans CLASSE_MODULE et non dans INSCRIPTIONS ?

Tous les apprenants d'une même classe avancent ensemble dans un module.

L'état 'En cours' ou 'Terminé' est une propriété collective de la classe sur un module.

Si etat était dans INSCRIPTIONS, rien n'empêcherait Moussa d'être 'Terminé' et Fatou 'Non commencé' dans la même classe.

En mettant etat dans CLASSE_MODULE, un seul enregistrement gère l'état pour toute la classe.

Pourquoi INSCRIPTIONS existe si CLASSE_MODULE définit déjà le programme ?

CLASSE_MODULE définit le programme collectif (quels modules une classe fait).

INSCRIPTIONS matérialise l'acte administratif individuel avec la date et le groupe.

Le groupe est une information individuelle : le prof peut mettre Ndeye Penda en groupe 1 pour SQL et groupe 2 pour Python.

Un apprenant rejoignant la classe en cours d'année aura une date d'inscription différente.

Ces deux tables répondent à deux questions différentes : 'Quel est le programme ?' vs 'Qui est inscrit quand et dans quel groupe ?'

Pourquoi NOTES est indépendante d'INSCRIPTIONS ?

Le sujet demande explicitement : 'Afficher les apprenants ayant une note mais non inscrits (anomalie)'.

Si NOTES était liée à INSCRIPTIONS via une FK, cette anomalie serait techniquement impossible.

En gardant NOTES indépendante, on peut détecter des incohérences de données.

C'est une approche réaliste : dans les systèmes en production, les anomalies existent et doivent être détectables.

3. Création de la Base de Données

3.1 Code SQL de création

```
mysql> CREATE DATABASE nps_plateforme_formation;
Query OK, 1 row affected (0,03 sec)

mysql> USE nps_plateforme_formation;
Database changed
```

```
mysql> CREATE TABLE classes (
  ->   id INT AUTO_INCREMENT PRIMARY KEY,
  ->   nom VARCHAR(50) NOT NULL UNIQUE
  -> );
Query OK, 0 rows affected (0,07 sec)
```

```
mysql> CREATE TABLE modules (
  ->   id INT AUTO_INCREMENT PRIMARY KEY,
  ->   nom VARCHAR(100) NOT NULL,
  ->   categorie VARCHAR(100) NOT NULL,
  ->   duree_heures DECIMAL(5,1) NOT NULL
  -> );
Query OK, 0 rows affected (0,06 sec)
  ->   ON DELETE RESTRICT ON UPDATE CASCADE,
  ->   FOREIGN KEY (module_id) REFERENCES modules(id)
  ->   ON DELETE RESTRICT ON UPDATE CASCADE
  -> );
Query OK, 0 rows affected (0,04 sec)
```

```
mysql> CREATE TABLE classe_module (  
->   classe_id INT NOT NULL,  
->   module_id INT NOT NULL,  
->   etat      ENUM('Non commencé', 'En cours', 'Terminé')  
->           NOT NULL DEFAULT 'Non commencé',  
->   PRIMARY KEY (classe_id, module_id),  
->   FOREIGN KEY (classe_id) REFERENCES classes(id)  
->       ON DELETE RESTRICT ON UPDATE CASCADE,  
->   FOREIGN KEY (module_id) REFERENCES modules(id)  
->       ON DELETE RESTRICT ON UPDATE CASCADE  
-> );  
Query OK, 0 rows affected (0,04 sec)
```

```
mysql> CREATE TABLE apprenants (  
->   id          INT          AUTO_INCREMENT PRIMARY KEY,  
->   nom         VARCHAR(100) NOT NULL,  
->   prenom      VARCHAR(100) NOT NULL,  
->   email       VARCHAR(150) NOT NULL UNIQUE,  
->   sexe        ENUM('M','F') NOT NULL,  
->   adresse     VARCHAR(255),  
->   classe_id  INT          NOT NULL,  
->   actif       BOOLEAN     NOT NULL DEFAULT TRUE,  
->   FOREIGN KEY (classe_id) REFERENCES classes(id)  
->       ON DELETE RESTRICT ON UPDATE CASCADE  
-> );  
Query OK, 0 rows affected (0,07 sec)
```

```
mysql> CREATE TABLE inscriptions (  
->   id                INT      AUTO_INCREMENT PRIMARY KEY,  
->   apprenant_id      INT      NOT NULL,  
->   module_id         INT      NOT NULL,  
->   date_inscription DATE NOT NULL,  
->   groupe            ENUM('G1','G2','G3','G4') NOT NULL,  
->   UNIQUE KEY uq_inscription (apprenant_id, module_id),  
->   FOREIGN KEY (apprenant_id) REFERENCES apprenants(id)  
->       ON DELETE RESTRICT ON UPDATE CASCADE,  
->   FOREIGN KEY (module_id) REFERENCES modules(id)  
->       ON DELETE RESTRICT ON UPDATE CASCADE  
-> );  
Query OK, 0 rows affected (0,05 sec)
```

```
mysql> CREATE TABLE notes (  
->   id                INT      AUTO_INCREMENT PRIMARY KEY,  
->   valeur            DECIMAL(5,2) NOT NULL,  
->   type_note         ENUM('Devoir','Projet','Examen') NOT NULL,  
->   apprenant_id      INT      NOT NULL,  
->   module_id         INT      NOT NULL,  
->   date_evaluation   DATE      NOT NULL,  
->   CONSTRAINT chk_valeur CHECK (valeur >= 0 AND valeur <= 20),  
->   FOREIGN KEY (apprenant_id) REFERENCES apprenants(id)  
->       ON DELETE RESTRICT ON UPDATE CASCADE,  
->   FOREIGN KEY (module_id) REFERENCES modules(id)  
->       ON DELETE RESTRICT ON UPDATE CASCADE  
-> );  
Query OK, 0 rows affected (0,07 sec)
```

4. Insertion des Données — Partie 2

4.1 Réponses aux questions

Question 1 — Comment insérer plusieurs lignes dans une table ?

On utilise un seul INSERT avec plusieurs tuples séparés par des virgules. C'est plus efficace car MySQL exécute une seule transaction au lieu de N transactions.

Question 2 — Comment garantir la cohérence des relations ?

En respectant l'ordre d'insertion : toujours insérer les tables parents avant les tables enfants.

Ordre	Table	Dépend de
1	CLASSES	Aucune dépendance
2	MODULES	Aucune dépendance
3	CLASSE_MODULE	CLASSES + MODULES
4	APPRENANTS	CLASSES
5	INSCRIPTIONS	APPRENANTS + MODULES
6	NOTES	APPRENANTS + MODULES

4.2 Données insérées — Logique et justification

Structure des données — Règles appliquées

3 classes : DEV-DATA (5 apprenants), DEV-WEB (3 apprenants), DEV-IA (2 apprenants).

6 modules dont Cybersécurité sans aucun apprenant (pour la question 'modules sans apprenant').

Tous les apprenants d'une même classe font les mêmes modules avec le même état.
 Les types d'évaluation respectent l'état : Terminé → Devoir+Examen, En cours → Projet.
 Seydina (id 11) : aucune inscription mais a une note → anomalie volontaire pour Q18.
 Pape (id 10) : exactement 1 module → pour Q12.
 DEV-IA fait Python mais pas SQL → pour Q16 (Python mais pas SQL).
 Ndeye Penda : groupe 1 en SQL, groupe 2 en Python → pour Q23 (deux groupes différents).

```
mysql> INSERT INTO classes (nom) VALUES
-> ('DEV-DATA'),
-> ('DEV-WEB'),
-> ('DEV-IA');
Query OK, 3 rows affected (0,01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO modules (nom, categorie, duree_heures) VALUES
-> ('SQL', 'Base de données', 40.0),
-> ('Python', 'Programmation', 60.0),
-> ('Power BI', 'Visualisation', 30.0),
-> ('Web', 'Développement', 50.0),
-> ('Machine Learning', 'Data Science', 45.0),
-> ('Cybersécurité', 'Sécurité', 35.0);
Query OK, 6 rows affected (0,02 sec)
Records: 6 Duplicates: 0 Warnings: 0

mysql> INSERT INTO classe_module (classe_id, module_id, etat) VALUES
-> (1, 1, 'Terminé'),
-> (1, 2, 'En cours'),
-> (1, 3, 'Non commencé'),
-> (2, 4, 'Terminé'),
-> (2, 1, 'En cours'),
-> (3, 2, 'Terminé'),
-> (3, 5, 'En cours');
Query OK, 7 rows affected (0,03 sec)
Records: 7 Duplicates: 0 Warnings: 0

mysql> INSERT INTO apprenants (nom, prenom, email, sexe, adresse, classe_id, actif) VALUES
-> ('SARR', 'Ndeye Penda', 'ndeye.sarr@odc.sn', 'F', 'Mbour', 1, TRUE),
-> ('DIALLO', 'Ousmane', 'ousmane.diallo@odc.sn', 'M', 'Parcelles Assainies', 1, TRUE),
-> ('DIABY', 'Moussa', 'moussa.diaby@odc.sn', 'M', 'Guédiawaye', 1, TRUE),
-> ('DJIGO', 'Abdou', 'abdou.djigo@odc.sn', 'M', 'Médina', 1, TRUE),
-> ('DIALLO', 'Mariama', 'mariama.diallo@odc.sn', 'F', 'Yoff', 1, TRUE),
-> ('SOW', 'Ibrahima', 'ibrahima.sow@odc.sn', 'M', 'Grand Yoff', 2, TRUE),
-> ('NDIAYE', 'Aminata', 'aminata.ndiaye@odc.sn', 'F', 'Ouakam', 2, TRUE),
-> ('FALL', 'Omar', 'omar.fall@odc.sn', 'M', 'Pikine', 2, TRUE),
-> ('MBAYE', 'Rokhaya', 'rokhaya.mbaye@odc.sn', 'F', 'Liberté 6', 3, TRUE),
-> ('GUEYE', 'Pape', 'pape.gueye@odc.sn', 'M', 'HLM', 3, TRUE),
-> ('WADE', 'Seydina', 'seydina.wade@odc.sn', 'M', 'Dakar', 1, TRUE);
Query OK, 11 rows affected (0,01 sec)
Records: 11 Duplicates: 0 Warnings: 0
```

```
mysql> INSERT INTO notes (valeur, type_note, apprenant_id, module_id, date_evaluation) VALUES
-> (14.00, 'Devoir', 1, 1, '2026-03-01'),
-> (15.00, 'Examen', 1, 1, '2026-03-15'),
-> (12.00, 'Devoir', 2, 1, '2026-03-01'),
-> (09.00, 'Examen', 2, 1, '2026-03-15'),
-> (16.00, 'Devoir', 3, 1, '2026-03-01'),
-> (15.00, 'Examen', 3, 1, '2026-03-15'),
-> (13.00, 'Devoir', 4, 1, '2026-03-01'),
-> (11.00, 'Examen', 4, 1, '2026-03-15'),
-> (17.00, 'Devoir', 5, 1, '2026-03-01'),
-> (16.00, 'Examen', 5, 1, '2026-03-15'),
-> (13.00, 'Projet', 1, 2, '2026-04-10'),
-> (11.00, 'Projet', 2, 2, '2026-04-10'),
-> (14.00, 'Projet', 3, 2, '2026-04-10'),
-> (08.00, 'Projet', 4, 2, '2026-04-10'),
-> (15.00, 'Projet', 5, 2, '2026-04-10'),
-> (13.00, 'Devoir', 6, 4, '2026-03-05'),
-> (14.00, 'Examen', 6, 4, '2026-03-20'),
-> (07.00, 'Devoir', 7, 4, '2026-03-05'),
-> (09.00, 'Examen', 7, 4, '2026-03-20'),
-> (11.00, 'Devoir', 8, 4, '2026-03-05'),
-> (10.00, 'Examen', 8, 4, '2026-03-20'),
-> (12.00, 'Devoir', 6, 1, '2026-04-15'),
-> (10.00, 'Devoir', 7, 1, '2026-04-15'),
-> (08.00, 'Devoir', 8, 1, '2026-04-15'),
-> (16.00, 'Devoir', 9, 2, '2026-03-10'),
-> (17.00, 'Examen', 9, 2, '2026-03-25'),
-> (14.00, 'Devoir', 10, 2, '2026-03-10'),
-> (15.00, 'Examen', 10, 2, '2026-03-25'),
-> (12.00, 'Projet', 9, 5, '2026-04-20'),
-> (12.00, 'Examen', 11, 1, '2026-03-15');
Query OK, 30 rows affected (0,04 sec)
Records: 30  Duplicates: 0  Warnings: 0
```

5. Algèbre Relationnelle et Requêtes SQL — Partie 3

Pour chaque question, nous précisons : le type de question, l'expression en algèbre relationnelle, l'opérateur utilisé, et la requête SQL correspondante.

5.1 Questions de base — Projection et Sélection

Q1 — Afficher tous les apprenants

Type	Algèbre relationnelle	Opérateur
Projection totale	π id, nom, prenom, email, sexe, adresse, classe_id, actif	Projection (π)

```
mysql> SELECT * FROM apprenants;
```

id	nom	prenom	email	sexe	adresse	classe_id	actif
1	SARR	Ndeye Penda	ndeye.sarr@odc.sn	F	Mbour	1	1
2	DIALLO	Ousmane	ousmane.diallo@odc.sn	M	Parcelles Assainies	1	1
3	DIABY	Moussa	moussa.diaby@odc.sn	M	Guédiawaye	1	1
4	DJIGO	Abdou	abdou.djigo@odc.sn	M	Médina	1	1
5	DIALLO	Mariama	maria.diallo@odc.sn	F	Yoff	1	1
6	SOW	Ibrahima	ibrahima.sow@odc.sn	M	Grand Yoff	2	1
7	NDIAYE	Aminata	aminata.ndiaye@odc.sn	F	Ouakam	2	1
8	FALL	Omar	omar.fall@odc.sn	M	Pikine	2	1
9	MBAYE	Rokhaya	rokhaya.mbaye@odc.sn	F	Liberté 6	3	1
10	GUEYE	Pape	pape.gueye@odc.sn	M	HLM	3	1
11	WADE	Seydina	seydina.wade@odc.sn	M	Dakar	1	1

```
11 rows in set (0,00 sec)
```

Q2 — Afficher les apprenants d'une classe donnée

Type	Algèbre relationnelle	Opérateur
Sélection + Projection	π id, nom, prenom, sexe, email, classe (σ classe='DEV-DATA' (APPRENANTS $\bowtie$ CLASSES))	Sélection (σ)

```
mysql> SELECT a.id as id , a.prenom as prenom, a.nom as nom, a.sexe as sexe, a.email as email FROM apprenants a
-> JOIN classes c ON a.classe_id = c.id
-> WHERE c.nom = 'DEV-DATA';
+-----+
| id | prenom      | nom   | sexe | email                |
+-----+
| 1 | Ndeye Penda | SARR  | F    | ndeye.sarr@odc.sn    |
| 2 | Ousmane     | DIALLO | M    | ousmane.diallo@odc.sn |
| 3 | Moussa      | DIABY  | M    | moussa.diaby@odc.sn  |
| 4 | Abdou       | DJIGO  | M    | abdou.djigo@odc.sn   |
| 5 | Mariama     | DIALLO | F    | mariama.diallo@odc.sn |
| 11 | Seydina     | WADE   | M    | seydina.wade@odc.sn  |
+-----+
6 rows in set (0,00 sec)
```

Q3 — Afficher uniquement les noms des apprenants

Type	Algèbre relationnelle	Opérateur
Projection pure	π nom(APPRENANTS)	Projection (π)

```
mysql> SELECT DISTINCT nom FROM apprenants;
+-----+
| nom   |
+-----+
| SARR  |
| DIALLO |
| DIABY  |
| DJIGO  |
| SOW    |
| NDIAYE |
| FALL   |
| MBAYE  |
| GUEYE  |
| WADE   |
+-----+
10 rows in set (0,02 sec)
```

Q4 — Afficher les modules disponibles

Type	Algèbre relationnelle	Opérateur
Projection totale	π id, nom, categorie, duree_heures (MODULES)	Projection (π)

```
mysql> SELECT * FROM modules;
+----+-----+-----+-----+
| id | nom          | categorie      | duree_heures |
+----+-----+-----+-----+
| 1  | SQL          | Base de données | 40.0          |
| 2  | Python       | Programmation  | 60.0          |
| 3  | Power BI     | Visualisation  | 30.0          |
| 4  | Web          | Développement  | 50.0          |
| 5  | Machine Learning | Data Science  | 45.0          |
| 6  | Cybersécurité | Sécurité       | 35.0          |
+----+-----+-----+-----+
6 rows in set (0,00 sec)
```

Q5 — Afficher les notes supérieures à une valeur donnée

Type	Algèbre relationnelle	Opérateur
Sélection	σ valeur > 12 (NOTES)	Sélection (σ)

```
mysql> SELECT * FROM notes WHERE valeur > 12;
```

id	valeur	type_note	apprenant_id	module_id	date_evaluation
1	14.00	Devoir	1	1	2026-03-01
2	15.00	Examen	1	1	2026-03-15
5	16.00	Devoir	3	1	2026-03-01
6	15.00	Examen	3	1	2026-03-15
7	13.00	Devoir	4	1	2026-03-01
9	17.00	Devoir	5	1	2026-03-01
10	16.00	Examen	5	1	2026-03-15
11	13.00	Projet	1	2	2026-04-10
13	14.00	Projet	3	2	2026-04-10
15	15.00	Projet	5	2	2026-04-10
16	13.00	Devoir	6	4	2026-03-05
17	14.00	Examen	6	4	2026-03-20
25	16.00	Devoir	9	2	2026-03-10
26	17.00	Examen	9	2	2026-03-25
27	14.00	Devoir	10	2	2026-03-10
28	15.00	Examen	10	2	2026-03-25

16 rows in set (0,00 sec)

5.2 Jointures

Q6 — Afficher les apprenants avec les modules auxquels ils sont inscrits

Type	Algèbre relationnelle	Opérateur
Jointure	π nom, prenom, sexe, module (APPRENANTS $\bowtie$ INSCRIPTIONS $\bowtie$ MODULES)	Jointure ($\bowtie$)

```
mysql> SELECT a.id as id , a.prenom as prenom, a.nom as nom, modules.nom , modules.categorie FROM apprenants a
-> JOIN inscriptions i ON a.id = i.apprenant_id
-> JOIN modules ON modules.id = i.module_id;
```

id	prenom	nom	nom	categorie
1	Ndeye Penda	SARR	SQL	Base de données
2	Ousmane	DIALLO	SQL	Base de données
3	Moussa	DIABY	SQL	Base de données
4	Abdou	DJIGO	SQL	Base de données
5	Mariama	DIALLO	SQL	Base de données
6	Ibrahima	SOW	SQL	Base de données
7	Aminata	NDIAYE	SQL	Base de données
8	Omar	FALL	SQL	Base de données
1	Ndeye Penda	SARR	Python	Programmation
2	Ousmane	DIALLO	Python	Programmation
3	Moussa	DIABY	Python	Programmation
4	Abdou	DJIGO	Python	Programmation
5	Mariama	DIALLO	Python	Programmation
9	Rokhaya	MBAYE	Python	Programmation
10	Pape	GUEYE	Python	Programmation
1	Ndeye Penda	SARR	Power BI	Visualisation
2	Ousmane	DIALLO	Power BI	Visualisation
3	Moussa	DIABY	Power BI	Visualisation
4	Abdou	DJIGO	Power BI	Visualisation
5	Mariama	DIALLO	Power BI	Visualisation
6	Ibrahima	SOW	Web	Développement
7	Aminata	NDIAYE	Web	Développement
8	Omar	FALL	Web	Développement
9	Rokhaya	MBAYE	Machine Learning	Data Science

24 rows in set (0,00 sec)

Q7 — Afficher les apprenants avec leurs notes

Type	Algèbre relationnelle	Opérateur
Jointure	π nom, prenom, module, valeur (APPRENANTS $\bowtie$ NOTES $\bowtie$ MODULES)	Jointure ($\bowtie$)

```
mysql> SELECT a.id as id , a.prenom as prenom, a.nom as nom , valeur , type_note, m.nom FROM apprenants a
-> JOIN notes n ON a.id = n.apprenant_id
-> JOIN modules m ON m.id = n.module_id;
```

id	prenom	nom	valeur	type_note	nom
1	Ndeye Penda	SARR	14.00	Devoir	SQL
1	Ndeye Penda	SARR	15.00	Examen	SQL
2	Ousmane	DIALLO	12.00	Devoir	SQL
2	Ousmane	DIALLO	9.00	Examen	SQL
3	Moussa	DIABY	16.00	Devoir	SQL
3	Moussa	DIABY	15.00	Examen	SQL
4	Abdou	DJIGO	13.00	Devoir	SQL
4	Abdou	DJIGO	11.00	Examen	SQL
5	Mariama	DIALLO	17.00	Devoir	SQL
5	Mariama	DIALLO	16.00	Examen	SQL
6	Ibrahima	SOW	12.00	Devoir	SQL
7	Aminata	NDIAYE	10.00	Devoir	SQL
8	Omar	FALL	8.00	Devoir	SQL
11	Seydina	WADE	12.00	Examen	SQL
1	Ndeye Penda	SARR	13.00	Projet	Python
2	Ousmane	DIALLO	11.00	Projet	Python
3	Moussa	DIABY	14.00	Projet	Python
4	Abdou	DJIGO	8.00	Projet	Python
5	Mariama	DIALLO	15.00	Projet	Python
9	Rokhaya	MBAYE	16.00	Devoir	Python
9	Rokhaya	MBAYE	17.00	Examen	Python
10	Pape	GUEYE	14.00	Devoir	Python
10	Pape	GUEYE	15.00	Examen	Python
6	Ibrahima	SOW	13.00	Devoir	Web
6	Ibrahima	SOW	14.00	Examen	Web
7	Aminata	NDIAYE	7.00	Devoir	Web
7	Aminata	NDIAYE	9.00	Examen	Web
8	Omar	FALL	11.00	Devoir	Web
8	Omar	FALL	10.00	Examen	Web
9	Rokhaya	MBAYE	12.00	Projet	Machine Learning

30 rows in set (0,00 sec)

Q8 — Afficher les apprenants inscrits dans un module donné

Type	Algèbre relationnelle	Opérateur
Sélection + Jointure	π nom, prenom , classe, nom_modules (σ module='Python' (APPRENANTS $\bowtie$ INSCRIPTIONS $\bowtie$ MODULES))	Sélection (σ) + Jointure ($\bowtie$)

```
mysql> SELECT a.id as id , a.prenom as prenom, a.nom as nom FROM apprenants a
-> JOIN inscriptions i ON a.id = i.apprenant_id
-> JOIN modules m ON m.id = i.module_id
-> WHERE m.nom = 'Python';
+----+-----+-----+
| id | prenom | nom |
+----+-----+-----+
| 1 | Ndeye Penda | SARR |
| 2 | Ousmane | DIALLO |
| 3 | Moussa | DIABY |
| 4 | Abdou | DJIGO |
| 5 | Mariama | DIALLO |
| 9 | Rokhaya | MBAYE |
| 10 | Pape | GUEYE |
+----+-----+-----+
7 rows in set (0,00 sec)
```

Q9 — Afficher les modules suivis par un apprenant donné

Type	Algèbre relationnelle	Opérateur
Sélection + Jointure	π module (σ prenom='Ndeye Penda' (APPRENANTS $\bowtie$ INSCRIPTIONS $\bowtie$ MODULES))	Sélection (σ) + Jointure ($\bowtie$)

```
mysql> SELECT m.id , m.nom, m.categorie FROM modules m
-> JOIN inscriptions i ON m.id = i.module_id
-> JOIN apprenants a ON a.id = i.apprenant_id
-> WHERE a.prenom = 'Ndeye Penda';
+----+-----+-----+
| id | nom | categorie |
+----+-----+-----+
| 1 | SQL | Base de données |
| 2 | Python | Programmation |
| 3 | Power BI | Visualisation |
+----+-----+-----+
3 rows in set (0,00 sec)
```

Q10 — Afficher les apprenants ayant une note supérieure à une valeur dans un module donné

Type	Algèbre relationnelle	Opérateur
Sélection + Jointure	$\pi \text{ nom, prenom } (\sigma \text{ module='SQL' } \wedge \text{ valeur} > 14 \text{ (APPRENANTS } \bowtie \text{ NOTES } \bowtie \text{ MODULES)})$	Sélection (σ) + Jointure ($\bowtie$)

```
mysql> SELECT a.id as id , a.prenom as prenom, a.nom as nom , valeur , type_note, m.nom FROM apprenants a
-> JOIN notes n ON a.id = n.apprenant_id
-> JOIN modules m ON m.id = n.module_id
-> WHERE m.nom = 'SQL' and valeur >= 15;
+-----+-----+-----+-----+-----+-----+
| id | prenom | nom | valeur | type_note | nom |
+-----+-----+-----+-----+-----+-----+
| 1 | Ndeye Penda | SARR | 15.00 | Examen | SQL |
| 3 | Moussa | DIABY | 16.00 | Devoir | SQL |
| 3 | Moussa | DIABY | 15.00 | Examen | SQL |
| 5 | Mariama | DIALLO | 17.00 | Devoir | SQL |
| 5 | Mariama | DIALLO | 16.00 | Examen | SQL |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0,00 sec)
```

5.3 Agrégation

Q11 — Afficher les apprenants inscrits dans au moins deux modules

Type	Algèbre relationnelle	Opérateur
Agrégation + Filtre	$\sigma \text{ nb_modules } \geq 2 (\gamma \text{ apprenant_id; COUNT(module_id)} \rightarrow \text{nb_modules(INSRIPTIONS)})$	Agrégation (γ) + Sélection (σ)

```
mysql> SELECT a.id, a.prenom, a.nom, COUNT(DISTINCT m.id) AS nbre_modules
-> FROM apprenants a
-> JOIN inscriptions i ON a.id = i.apprenant_id
-> JOIN modules m ON m.id = i.module_id
-> GROUP BY a.id, a.prenom, a.nom
-> HAVING COUNT(DISTINCT m.id) >= 2;
+----+-----+-----+-----+
| id | prenom | nom | nbre_modules |
+----+-----+-----+-----+
| 1 | Ndeye Penda | SARR | 3 |
| 2 | Ousmane | DIALLO | 3 |
| 3 | Moussa | DIABY | 3 |
| 4 | Abdou | DJIGO | 3 |
| 5 | Mariama | DIALLO | 3 |
| 6 | Ibrahima | SOW | 2 |
| 7 | Aminata | NDIAYE | 2 |
| 8 | Omar | FALL | 2 |
| 9 | Rokhaya | MBAYE | 2 |
+----+-----+-----+-----+
9 rows in set (0,02 sec)
```

Q12 — Afficher les apprenants inscrits dans exactement un module

Type	Algèbre relationnelle	Opérateur
Agrégation + Filtre exact	$\sigma_{nb_modules = 1}(\gamma_{apprenant_id ;$ $COUNT(module_id) \rightarrow$ $nb_modules(INSCRIPTIONS$ $))$	Agrégation (γ)

```
mysql> SELECT a.id, a.prenom, a.nom, COUNT(DISTINCT m.id) AS nbre_modules
-> FROM apprenants a
-> JOIN inscriptions i ON a.id = i.apprenant_id
-> JOIN modules m ON m.id = i.module_id
-> GROUP BY a.id, a.prenom, a.nom
-> HAVING COUNT(DISTINCT m.id) = 1;
+-----+-----+-----+-----+
| id | prenom | nom   | nbre_modules |
+-----+-----+-----+-----+
| 10 | Pape   | GUEYE | 1            |
+-----+-----+-----+-----+
1 row in set (0,00 sec)
```

5.4 Différence

Q13 — Afficher les apprenants qui ne sont inscrits dans aucun module

Type	Algèbre relationnelle	Opérateur
Différence	$\pi \text{ nom (APPRENANTS)} - \pi$ $\text{nom (APPRENANTS} \bowtie$ INSCRIPTIONS)	Différence (−)

```
mysql> SELECT a.id, a.prenom, a.nom
-> FROM apprenants a
-> LEFT JOIN inscriptions i ON a.id = i.apprenant_id
-> WHERE i.id IS NULL;
+-----+-----+-----+
| id | prenom | nom   |
+-----+-----+-----+
| 11 | Seydina | WADE |
+-----+-----+-----+
1 row in set (0,00 sec)
```

Q14 — Afficher les modules qui n'ont aucun apprenant

Type	Algèbre relationnelle	Opérateur
Différence	$\pi \text{ nom (MODULES)} - \pi \text{ nom (MODULES} \bowtie \text{INSCRIPTIONS)}$	Différence (-)

```
mysql> SELECT m.id, m.nom, m.categorie
-> FROM modules m
-> LEFT JOIN inscriptions i ON m.id = i.module_id
-> WHERE i.id IS NULL;
+----+-----+-----+
| id | nom           | categorie |
+----+-----+-----+
| 6  | Cybersécurité | Sécurité  |
+----+-----+-----+
1 row in set (0,00 sec)
```

Q15 — Afficher les apprenants ayant au moins une note

Type	Algèbre relationnelle	Opérateur
Jointure + Dédoublonnage	$\pi \text{ nom, prenom (APPRENANTS} \bowtie \text{NOTES)}$	Jointure ($\bowtie$)

```
mysql> SELECT DISTINCT a.id,a.prenom, a.nom
-> FROM apprenants a
-> JOIN notes n ON a.id = n.apprenant_id;
+----+-----+-----+
| id | prenom      | nom    |
+----+-----+-----+
| 1  | Ndeye Penda | SARR   |
| 2  | Ousmane     | DIALLO |
| 3  | Moussa      | DIABY  |
| 4  | Abdou       | DJIGO  |
| 5  | Mariama     | DIALLO |
| 6  | Ibrahima    | SOW    |
| 7  | Aminata     | NDIAYE |
| 8  | Omar        | FALL   |
| 9  | Rokhaya     | MBAYE  |
| 10 | Pape        | GUEYE  |
| 11 | Seydina     | WADE   |
+----+-----+-----+
11 rows in set (0,00 sec)
```

Q16 — Afficher les apprenants inscrits en Python mais pas en SQL

Type	Algèbre relationnelle	Opérateur
Différence	$\pi \text{ nom } (\sigma \text{ module='Python'}(...)) - \pi \text{ nom } (\sigma \text{ module='SQL'}(...))$	Différence (-)

```
mysql> SELECT a.id as id , a.prenom as prenom, a.nom as nom FROM apprenants a
-> JOIN inscriptions i ON a.id = i.apprenant_id
-> JOIN modules m ON m.id = i.module_id
-> WHERE m.nom = 'Python'
-> AND a.id NOT IN
-> (
->     SELECT a1.id FROM apprenants a1
->     JOIN inscriptions i1 ON a1.id = i1.apprenant_id
->     JOIN modules m1 ON m1.id = i1.module_id
->     WHERE m1.nom = 'SQL'
-> );
+-----+-----+-----+
| id | prenom | nom |
+-----+-----+-----+
| 9 | Rokhaya | MBAYE |
| 10 | Pape | GUEYE |
+-----+-----+-----+
2 rows in set (0,00 sec)
```

Q17 — Afficher les apprenants inscrits mais n'ayant aucune note

Type	Algèbre relationnelle	Opérateur
Différence	$\pi \text{ nom } (\text{APPRENANTS} \bowtie \text{INSCRIPTIONS}) - \pi \text{ nom } (\text{APPRENANTS} \bowtie \text{NOTES})$	Différence (−)

```
mysql> SELECT DISTINCT a.id, a.prenom, a.nom
-> FROM apprenants a
-> JOIN inscriptions i ON a.id = i.apprenant_id
-> WHERE a.id NOT IN (SELECT DISTINCT apprenant_id FROM notes);
Empty set (0,00 sec)
```

Q18 — Afficher les apprenants ayant une note mais non inscrits — anomalie

Type	Algèbre relationnelle	Opérateur
Différence	$\pi \text{ nom, prenom (APPRENANTS} \bowtie (\pi \text{ apprenant_id,module_id(NOTES)} - \pi \text{ apprenant_id,module_id(INSRIPTIONS))))$	Différence (-)

```
mysql> SELECT a.id,a.prenom, a.nom
-> FROM apprenants a
-> JOIN notes n ON a.id = n.apprenant_id
-> LEFT JOIN inscriptions i ON a.id = i.apprenant_id AND n.module_id = i.module_id
-> WHERE i.id IS NULL;
+-----+-----+-----+
| id | prenom | nom |
+-----+-----+-----+
| 11 | Seydina | WADE |
+-----+-----+-----+
1 row in set (0,00 sec)
```

Q19 — Afficher les modules sans note associée

Type	Algèbre relationnelle	Opérateur
Différence	$\pi \text{ nom (MODULES)} - \pi \text{ nom (MODULES} \bowtie \text{NOTES)}$	Différence (-)

```
mysql> SELECT m.id as id , m.nom as nom, m.categorie as categorie FROM modules m
-> LEFT JOIN notes n ON m.id = n.module_id
-> WHERE n.id is NULL;
+-----+-----+-----+
| id | nom | categorie |
+-----+-----+-----+
| 3 | Power BI | Visualisation |
| 6 | Cybersécurité | Sécurité |
+-----+-----+-----+
2 rows in set (0,00 sec)
```

Q20 — Afficher les apprenants qui n'ont pas validé un module donné

Type	Algèbre relationnelle	Opérateur
Agrégation + Sélection	$\sigma_{\text{moyenne} < 10} (\gamma_{\text{apprenant_id}, \text{module_id}}; \text{AVG}(\text{valeur}) \rightarrow \text{moyenne} (\text{APPRENANTS} \bowtie \text{NOTES} \bowtie \text{MODULES}))$	Agrégation + Sélection

```
mysql> SELECT a.id,a.prenom,a.nom, AVG(n.valeur) AS moyenne_module, m.nom AS module FROM apprenants a
-> JOIN notes n ON a.id = n.apprenant_id
-> JOIN modules m ON m.id = n.module_id
-> WHERE m.nom = 'Python'
-> GROUP BY a.id, a.prenom, a.nom, m.nom
-> HAVING AVG(n.valeur) < 10;
+-----+-----+-----+-----+-----+
| id | prenom | nom   | moyenne_module | module |
+-----+-----+-----+-----+-----+
| 4  | Abdou  | DJIGO | 8.000000       | Python |
+-----+-----+-----+-----+-----+
1 row in set (0,00 sec)
```

5.5 Intersection**Q21 — Afficher les apprenants inscrits à la fois en Python et en SQL**

Type	Algèbre relationnelle	Opérateur
Intersection	$\pi_{\text{nom}} (\sigma_{\text{module}='Python'}(\dots)) \cap \pi_{\text{nom}} (\sigma_{\text{module}='SQL'}(\dots))$	Intersection ($\cap$)

```
mysql> SELECT a.id as id , a.prenom as prenom, a.nom as nom FROM apprenants a
-> JOIN inscriptions i ON a.id = i.apprenant_id
-> JOIN modules m ON m.id = i.module_id
-> WHERE m.nom = 'Python'
-> AND a.id IN
-> (
->     SELECT a1.id FROM apprenants a1
->     JOIN inscriptions i1 ON a1.id = i1.apprenant_id
->     JOIN modules m1 ON m1.id = i1.module_id
->     WHERE m1.nom = 'SQL'
-> );
```

id	prenom	nom
1	Ndeye Penda	SARR
2	Ousmane	DIALLO
3	Moussa	DIABY
4	Abdou	DJIGO
5	Mariama	DIALLO

5 rows in set (0,00 sec)

Q22 — Afficher les apprenants ayant une note dans deux modules donnés

Type	Algèbre relationnelle	Opérateur
Intersection	$\pi \text{ nom } (\sigma \text{ module='SQL'}(\text{NOTES})) \cap \pi \text{ nom } (\sigma \text{ module='Python'}(\text{NOTES}))$	Intersection ($\cap$)

```
mysql> SELECT a.id as id , a.prenom as prenom, a.nom as nom FROM apprenants a
-> JOIN notes n ON a.id = n.apprenant_id
-> JOIN modules m ON m.id = n.module_id
-> WHERE m.nom = 'Python'
-> AND a.id IN
-> (
->     SELECT  a1.id FROM apprenants a1
->     JOIN notes n1 ON a1.id = n1.apprenant_id
->     JOIN modules m1 ON m1.id = n1.module_id
->     WHERE m1.nom = 'SQL'
-> );
```

id	prenom	nom
1	Ndeye Penda	SARR
2	Ousmane	DIALLO
3	Moussa	DIABY
4	Abdou	DJIGO
5	Mariama	DIALLO

5 rows in set (0,00 sec)

Q23 — Afficher les apprenants présents dans deux groupes différents

Type	Algèbre relationnelle	Opérateur
Agrégation + Filtre	σ nb_groupes ≥ 2 (γ apprenant_id ;COUNT(DISTINCT groupe) $\rightarrow$ nb_groupes (INSCRIPTIONS))	Agrégation (γ) + Sélection (σ)

```
mysql> SELECT a.id as id , a.prenom as prenom, a.nom as nom , COUNT(DISTINCT i.groupe) as nbre_groupes_distincts FROM apprenants a
-> JOIN inscriptions i ON a.id = i.apprenant_id
-> JOIN modules m ON m.id = i.module_id
-> GROUP BY a.id, a.prenom, a.nom
-> HAVING nbre_groupes_distincts >= 2;
+-----+-----+-----+-----+
| id | prenom | nom | nbre_groupes_distincts |
+-----+-----+-----+-----+
| 1 | Ndeye Penda | SARR | 2 |
| 2 | Ousmane | DIALLO | 2 |
| 3 | Moussa | DIABY | 2 |
| 4 | Abdou | DJIGO | 2 |
| 5 | Mariama | DIALLO | 2 |
| 6 | Ibrahima | SOW | 2 |
| 7 | Aminata | NDIAYE | 2 |
| 8 | Omar | FALL | 2 |
+-----+-----+-----+-----+
8 rows in set (0,01 sec)
```

5.6 Union

Q24 — Afficher les apprenants inscrits en Python ou en Web

Type	Algèbre relationnelle	Opérateur
Union	$\pi \text{ nom } (\sigma$ $\text{module='Python'}(...)) \cup \pi$ $\text{nom } (\sigma \text{ module='Web'}(...))$	Union (U)

```
mysql> SELECT DISTINCT a.id,a.prenom,a.nom FROM apprenants a
-> JOIN inscriptions i ON a.id = i.apprenant_id
-> JOIN modules m ON m.id = i.module_id
-> WHERE m.nom IN ('Python','Web');

+----+-----+-----+
| id | prenom      | nom    |
+----+-----+-----+
| 1  | Ndeye Penda | SARR   |
| 2  | Ousmane     | DIALLO |
| 3  | Moussa      | DIABY  |
| 4  | Abdou       | DJIGO  |
| 5  | Mariama     | DIALLO |
| 9  | Rokhaya     | MBAYE  |
| 10 | Pape        | GUEYE  |
| 6  | Ibrahima    | SOW    |
| 7  | Aminata     | NDIAYE |
| 8  | Omar        | FALL   |
+----+-----+-----+
10 rows in set (0,01 sec)
```

Q25 — Afficher tous les apprenants ayant une note ou une inscription

Type	Algèbre relationnelle	Opérateur
Union	$\pi_{\text{nom}} (\text{APPRENANTS} \bowtie \text{NOTES}) \cup \pi_{\text{nom}} (\text{APPRENANTS} \bowtie \text{INSCRIPTIONS})$	Union ($\cup$)

```
mysql> SELECT DISTINCT a.id, a.prenom, a.nom
-> FROM apprenants a
-> JOIN notes n ON a.id = n.apprenant_id
-> UNION
-> SELECT DISTINCT a.id, a.prenom, a.nom
-> FROM apprenants a
-> JOIN inscriptions i ON a.id = i.apprenant_id;
+----+-----+-----+
| id | prenom      | nom    |
+----+-----+-----+
| 1  | Ndeye Penda | SARR   |
| 2  | Ousmane     | DIALLO |
| 3  | Moussa      | DIABY  |
| 4  | Abdou       | DJIGO  |
| 5  | Mariama     | DIALLO |
| 6  | Ibrahima    | SOW    |
| 7  | Aminata     | NDIAYE |
| 8  | Omar        | FALL   |
| 9  | Rokhaya     | MBAYE  |
| 10 | Pape        | GUEYE  |
| 11 | Seydina     | WADE   |
+----+-----+-----+
11 rows in set (0,00 sec)
```

5.7 Division

Q26 — Afficher les apprenants inscrits à tous les modules

Type	Algèbre relationnelle	Opérateur
Division	π apprenant_id,module_id (INSCRIPTIONS) $\div$ π id(MODULES)	Division ($\div$)

```
mysql> SELECT a.id as id , a.prenom as prenom, a.nom as nom , count(DISTINCT m.id) as nbre_modules FROM apprenants a
-> JOIN inscriptions i ON a.id = i.apprenant_id
-> JOIN modules m ON m.id = i.module_id
-> GROUP BY a.id , a.prenom, a.nom
-> HAVING nbre_modules = (SELECT COUNT(DISTINCT m.id) FROM modules m);
Empty set (0,00 sec)
```

Q27 — Afficher les apprenants ayant une note dans tous les modules

Type	Algèbre relationnelle	Opérateur
Division	$\pi_{\text{apprenant_id,module_id}}(\text{NOTES}) \div \pi_{\text{id}}(\text{MODULES})$	Division ($\div$)

```
mysql> SELECT a.id as id , a.prenom as prenom, a.nom as nom , count(DISTINCT n.module_id) as nbre_notes FROM apprenants a
-> JOIN notes n ON a.id = n.apprenant_id
-> JOIN modules m ON m.id = n.module_id
-> GROUP BY a.id , a.prenom, a.nom
-> HAVING nbre_notes= (SELECT COUNT(*) FROM modules);
Empty set (0,01 sec)
```

Q28 — Afficher les apprenants ayant validé tous les modules (note ≥ 10)

Type	Algèbre relationnelle	Opérateur
Division + Sélection	$\pi_{\text{apprenant_id,module_id}}(\sigma_{\text{valeur} \geq 10}(\text{NOTES})) \div \pi_{\text{id}}(\text{MODULES})$	Division ($\div$) + Sélection (σ)

```
mysql> SELECT a.id,a.prenom,a.nom FROM apprenants a
-> JOIN notes n ON a.id = n.apprenant_id
-> GROUP BY a.id, a.prenom, a.nom
-> HAVING COUNT(DISTINCT n.module_id) =
-> (
->     SELECT COUNT(*)
->     FROM modules
-> )
-> AND MIN(n.valeur) >= 10;
Empty set (0,03 sec)
```

Q29 — Afficher les apprenants inscrits dans tous modules d'une catégorie donnée

Type	Algèbre relationnelle	Opérateur
Division + Sélection	$\pi_{\text{apprenant_id}, \text{module_id}}(\text{INSCRIPTIONS}) \div \pi_{\text{id}}(\sigma_{\text{categorie}='Visualisation'}(\text{MODULES}))$	Division ($\div$)

```
mysql> SELECT a.id as id , a.prenom as prenom, a.nom as nom FROM apprenants a
-> JOIN inscriptions i ON a.id = i.apprenant_id
-> JOIN modules m ON m.id = i.module_id
-> WHERE m.categorie = 'Visualisation'
-> GROUP BY a.id , a.prenom, a.nom
-> HAVING count(DISTINCT m.id) = (SELECT COUNT(*) FROM modules m WHERE categorie = 'Visualisation');
+-----+-----+-----+
| id | prenom      | nom    |
+-----+-----+-----+
| 1 | Ndeye Penda | SARR   |
| 2 | Ousmane     | DIALLO |
| 3 | Moussa      | DIABY  |
| 4 | Abdou       | DJIGO  |
| 5 | Mariama     | DIALLO |
+-----+-----+-----+
5 rows in set (0,00 sec)
```

5.8 Agrégations avancées

Q30 — Calculer la moyenne des notes par apprenant

Type	Algèbre relationnelle	Opérateur
Agrégation	$\gamma_{\text{apprenant_id}}; \text{AVG}(\text{valeur})$ $(\text{NOTES} \bowtie \text{APPRENANTS})$	Agrégation (γ)

```
mysql> SELECT a.id as id , a.prenom as prenom, a.nom as nom , AVG(valeur) as moyenne FROM apprenants a
-> JOIN notes n ON a.id = n.apprenant_id
-> JOIN modules m ON m.id = n.module_id
-> GROUP BY a.id, a.prenom, a.nom;
+-----+-----+-----+-----+
| id | prenom      | nom   | moyenne |
+-----+-----+-----+-----+
| 1  | Ndeye Penda | SARR  | 14.000000 |
| 2  | Ousmane     | DIALLO | 10.666667 |
| 3  | Moussa      | DIABY  | 15.000000 |
| 4  | Abdou       | DJIGO  | 10.666667 |
| 5  | Mariama     | DIALLO | 16.000000 |
| 6  | Ibrahima    | SOW    | 13.000000 |
| 7  | Aminata     | NDIAYE | 8.666667  |
| 8  | Omar        | FALL   | 9.666667  |
| 9  | Rokhaya     | MBAYE  | 15.000000 |
| 10 | Pape        | GUEYE  | 14.500000 |
| 11 | Seydina     | WADE   | 12.000000 |
+-----+-----+-----+-----+
11 rows in set (0,00 sec)
```

Q31 — Afficher les apprenants ayant une moyenne supérieure à une valeur donnée

Type	Algèbre relationnelle	Opérateur
Agrégation + Filtre	σ moyenne > 15(γ apprenant_id ; AVG(valeur)→moyenne(NOTES))	Agrégation (γ) + Sélection (σ)

```
mysql> SELECT a.id as id , a.prenom as prenom, a.nom as nom , AVG(valeur) as moyenne FROM apprenants a
-> JOIN notes n ON a.id = n.apprenant_id
-> JOIN modules m ON m.id = n.module_id
-> GROUP BY a.id, a.prenom, a.nom
-> HAVING moyenne > 15;
+-----+-----+-----+-----+
| id | prenom | nom   | moyenne |
+-----+-----+-----+-----+
| 5  | Mariama | DIALLO | 16.000000 |
+-----+-----+-----+-----+
1 row in set (0,00 sec)
```

Q32 — Afficher le nombre d'apprenants par module

Type	Algèbre relationnelle	Opérateur
Agrégation	γ module_id ; COUNT(apprenant_id) $\rightarrow$ nb_apprenants (INSCRIPTIONS)	Agrégation (γ)

```
mysql> SELECT m.nom, count(a.id) as nbre_apprenants FROM apprenants a
-> JOIN inscriptions i ON a.id = i.apprenant_id
-> JOIN modules m ON m.id = i.module_id
-> GROUP BY m.nom;
```

```
+-----+-----+
| nom          | nbre_apprenants |
+-----+-----+
| SQL          | 8               |
| Python       | 7               |
| Power BI     | 5               |
| Web          | 3               |
| Machine Learning | 1               |
+-----+-----+
5 rows in set (0,00 sec)
```

Q33 — Afficher le module avec le plus d'apprenants

Type	Algèbre relationnelle	Opérateur
------	-----------------------	-----------

```
mysql> SELECT m.nom, count(a.id) as nbre_apprenants FROM apprenants a
-> JOIN inscriptions i ON a.id = i.apprenant_id
-> JOIN modules m ON m.id = i.module_id
-> GROUP BY m.nom
-> ORDER BY nbre_apprenants DESC LIMIT 1;
```

```
+-----+-----+
| nom | nbre_apprenants |
+-----+-----+
| SQL | 8               |
+-----+-----+
1 row in set (0,00 sec)
```

Agrégation + Maximum	$\sigma \text{ COUNT}=\text{MAX}(\text{COUNT}) (\gamma \text{ module ; COUNT}(\text{apprenant}) (\text{INSCRIPTIONS}))$	Agrégation (γ) + Maximum
----------------------	---	-----------------------------------

Q34 — Afficher l'apprenant ayant la meilleure moyenne

Type	Algèbre relationnelle	Opérateur
Agrégation + Maximum	$\sigma \text{ AVG}=\text{MAX}(\text{AVG}) (\gamma \text{ apprenant ; AVG}(\text{valeur}) (\text{NOTES}))$	Agrégation (γ) + Maximum

```
mysql> SELECT a.id as id , a.prenom as prenom, a.nom as nom , AVG(valeur) as moyenne FROM apprenants a
-> JOIN notes n ON a.id = n.apprenant_id
-> JOIN modules m ON m.id = n.module_id
-> GROUP BY a.id, a.prenom, a.nom
-> ORDER BY AVG(valeur) DESC LIMIT 1;
+-----+-----+-----+-----+
| id | prenom | nom   | moyenne |
+-----+-----+-----+-----+
| 5  | Mariama | DIALLO | 16.000000 |
+-----+-----+-----+-----+
1 row in set (0,01 sec)
```

Q35 — Afficher les apprenants dont toutes les notes sont supérieures à 10

Type	Algèbre relationnelle	Opérateur
Différence	$\pi \text{ nom} (\text{APPRENANTS} \bowtie \text{NOTES}) - \pi \text{ nom} (\sigma \text{ valeur} \leq 10 (\text{APPRENANTS} \bowtie \text{NOTES}))$	Différence (-)

```
mysql> SELECT a.id as id , a.prenom as prenom, a.nom as nom FROM apprenants a
-> JOIN notes n ON a.id = n.apprenant_id
-> JOIN modules m ON m.id = n.module_id
-> GROUP BY a.id, a.prenom, a.nom
-> HAVING min(valeur) >= 10;
```

id	prenom	nom
1	Ndeye Penda	SARR
3	Moussa	DIABY
5	Mariama	DIALLO
6	Ibrahima	SOW
9	Rokhaya	MBAYE
10	Pape	GUEYE
11	Seydina	WADE

7 rows in set (0,00 sec)

6.2 Ce que ce projet m'a appris

Leçons clés retenues

1. L'analyse du contexte métier est plus importante que le code. Sans comprendre 'établissement vs plateforme en ligne', le modèle aurait été faux.
2. La modélisation est itérative. Nous avons revu le modèle plusieurs fois : état dans INSCRIPTIONS → dans CLASSE_MODULE → ajout de la table CLASSES.
3. Chaque table doit avoir une raison d'être claire. INSCRIPTIONS et CLASSE_MODULE répondent à deux questions distinctes.
4. ON DELETE RESTRICT protège la traçabilité. Dans un établissement, on ne supprime jamais — on désactive (soft delete).
5. Les anomalies dans les données sont réelles. NOTES indépendant d'INSCRIPTIONS permet de les détecter.
6. L'algèbre relationnelle est le raisonnement derrière chaque requête SQL.
7. En production, un TRIGGER renforcerait la vérification module/classe. Pour ce projet, c'est géré applicativement.

6.3 Limites et améliorations possibles

- Ajouter un TRIGGER BEFORE INSERT sur INSCRIPTIONS pour vérifier que le module fait partie du programme de la classe de l'apprenant.
- Ajouter une table PLANNING pour gérer les dates de début et fin de chaque module par classe.
- Créer des VUES SQL pour les requêtes fréquentes (vue_notes_par_apprenant, vue_moyenne_par_classe).
- Implémenter un système de rôles (GRANT/REVOKE) pour sécuriser l'accès aux données selon le profil.